

ECHO: The Living World

Technical Design Document

Version 1.0

Project Name: ECHO: The Living World
Document: Technical Design Document (TDD)
Author: Bandhike Sai Vamshi
Institution: Dr. K. V. Subba Reddy Institute of Technology
Department: Artificial Intelligence and Machine Learning
Date: July 21, 2026
Status: Draft

Every choice leaves an Echo.

Copyright

© 2026 Bandhike Sai Vamshi.

All rights reserved.

This Technical Design Document is part of the original game project *ECHO: The Living World*.

No part of this document may be reproduced, distributed, or transmitted in any form or by any means without prior written permission from the author, except for brief quotations used for academic or review purposes.

This document contains original concepts, software architecture, gameplay systems, technical designs, and implementation plans created for the development of *ECHO: The Living World*.

The information contained in this document is intended solely for development, documentation, and educational purposes.

Version History

Version	Date	Description
1.0	July 21, 2026	Initial release of the Technical Design Document.

Document Status:

- Version: 1.0
- Status: Draft
- Author: Bandhike Sai Vamshi
- Repository: Project_ECHO

Contents

1	Introduction	1
1.1	Purpose	1
1.2	Objectives	1
1.3	Scope	2
1.4	Target Audience	2
1.5	Technology Stack	2
1.6	Engineering Philosophy	3
1.7	Summary	3
2	System Architecture	4
2.1	Overview	4
2.2	Architecture Layers	4
2.3	Core Modules	5
2.4	Scalability	5
2.5	Summary	5
3	Project Structure	6
3.1	Overview	6
3.2	Repository Structure	6
3.3	Unreal Source Structure	6
3.4	Naming Conventions	7
3.5	Summary	7
4	Coding Standards	8
4.1	Purpose	8
4.2	General Principles	8
4.3	Code Style	8
4.4	Documentation	9
4.5	Version Control	9
4.6	Summary	9
5	Core Gameplay Systems	10

5.1	Overview	10
5.2	Primary Systems	10
5.3	System Interaction	11
5.4	Design Goals	11
5.5	Summary	11
6	Player System	12
6.1	Overview	12
6.2	Responsibilities	12
6.3	Subsystems	13
6.4	Player States	13
6.5	Extensibility	14
6.6	Summary	14
7	AI System	15
7.1	Overview	15
7.2	Architecture	15
7.3	AI Categories	15
7.4	Perception	16
7.5	Behavior	16
7.6	Summary	16
8	Combat System	17
8.1	Overview	17
8.2	Components	17
8.3	Combat Actions	17
8.4	Damage Types	18
8.5	Design Goals	18
8.6	Summary	18
9	Inventory System	19
9.1	Overview	19
9.2	Inventory Categories	19
9.3	Operations	19
9.4	Data Storage	20
9.5	Summary	20
10	Quest System	21
10.1	Overview	21
10.2	Quest Types	21

10.3 Quest Structure	21
10.4 Quest States	22
10.5 Design Goals	22
10.6 Summary	22
11 Dialogue System	23
11.1 Overview	23
11.2 Core Components	23
11.3 Dialogue Flow	23
11.4 Dialogue Features	24
11.5 Design Goals	24
11.6 Summary	24
12 Save and Persistence	25
12.1 Overview	25
12.2 Saved Data	25
12.3 Save Types	26
12.4 Persistence Strategy	26
12.5 Data Integrity	26
12.6 Summary	26
13 Performance and Optimization	27
13.1 Overview	27
13.2 Performance Goals	27
13.3 Optimization Techniques	27
13.4 Profiling	28
13.5 Design Goals	28
13.6 Summary	28
14 Future Expansion	29
14.1 Overview	29
14.2 Expansion Opportunities	29
14.3 Scalability Principles	29
14.4 Long-Term Vision	30
14.5 Summary	30
15 Appendix	31
15.1 Abbreviations	31
15.2 Referenced Documents	31
15.3 Glossary	31

15.4 Document History	32
---------------------------------	----

Chapter 1

Introduction

1.1 Purpose

The Technical Design Document (TDD) provides the engineering blueprint for the development of *ECHO: The Living World*. While the Game Design Document (GDD) defines the creative vision, gameplay mechanics, and player experience, this document specifies how those systems are implemented from a software engineering perspective.

The TDD serves as a technical reference for programmers, technical designers, AI engineers, gameplay engineers, tools programmers, and future contributors. It establishes consistent development standards, system architecture, coding conventions, and implementation guidelines throughout the project's lifecycle.

1.2 Objectives

The primary objectives of this document are:

- Define the overall software architecture.
- Document the interaction between gameplay systems.
- Standardize coding practices.
- Describe core gameplay implementation.
- Establish scalable engineering principles.
- Simplify long-term maintenance and expansion.

1.3 Scope

This document covers the complete technical architecture of the project including gameplay systems, AI systems, save management, quest architecture, dialogue framework, inventory management, performance optimization, and future scalability.

Art production, world lore, and narrative design are documented separately within their respective project documentation.

1.4 Target Audience

This document is intended for:

- Gameplay Programmers
- AI Engineers
- Technical Designers
- Unreal Engine Developers
- Software Architects
- Future Contributors

1.5 Technology Stack

The planned technology stack includes:

- Unreal Engine 5
- C++
- Blueprint Visual Scripting
- Git and GitHub
- Visual Studio 2022
- Blender
- LaTeX Documentation

1.6 Engineering Philosophy

The technical architecture of *ECHO: The Living World* is based on the following engineering principles:

- Modular system design
- High cohesion and low coupling
- Reusable components
- Scalability
- Maintainability
- Performance-first development

Each gameplay feature should function as an independent module while remaining fully integrated with the overall game architecture.

1.7 Summary

The Technical Design Document acts as the engineering foundation of the project. It transforms the creative concepts defined in the Game Design Document into structured, implementable software systems that can evolve throughout development.

Chapter 2

System Architecture

2.1 Overview

The software architecture of *ECHO: The Living World* follows a modular layered architecture designed to maximize scalability, maintainability, and performance. Individual systems operate independently while communicating through clearly defined interfaces.

The architecture separates gameplay logic, rendering, artificial intelligence, physics, user interface, and data management into dedicated subsystems.

2.2 Architecture Layers

The engine is organized into the following major layers:

- Presentation Layer
- Gameplay Layer
- Artificial Intelligence Layer
- Physics Layer
- Audio Layer
- Data Layer
- Rendering Layer

Each layer has a clearly defined responsibility and minimizes dependencies on unrelated systems.

2.3 Core Modules

Major software modules include:

- Player System
- World Manager
- AI Manager
- Combat Manager
- Quest Manager
- Dialogue Manager
- Inventory Manager
- Save Manager
- Audio Manager
- UI Manager

These modules communicate through event-driven interactions rather than direct dependencies wherever possible.

2.4 Scalability

The architecture is designed to support:

- Future downloadable content
- Additional continents
- Expanded AI behaviors
- New gameplay mechanics
- Multiplayer integration (future consideration)

2.5 Summary

A modular architecture improves maintainability, testing, and future expansion while reducing development complexity.

Chapter 3

Project Structure

3.1 Overview

The project follows a modular directory organization to separate gameplay systems, assets, tools, and documentation.

3.2 Repository Structure

The repository contains the following primary components:

- Game Design Document
- Technical Design Document
- Narrative Bible
- Character Bible
- Art Bible
- Programming Guide
- Unreal Engine Project

3.3 Unreal Source Structure

The source code is organized into feature-based modules:

- Core
- Characters
- AI

- Combat
- Inventory
- Quests
- Dialogue
- Save
- UI
- World
- Utilities

3.4 Naming Conventions

General naming conventions include:

- PascalCase for classes
- camelCase for variables
- UPPER_SNAKE_CASE for constants
- Prefix Unreal classes according to engine standards

3.5 Summary

A consistent project structure improves readability, collaboration, and long-term maintainability.

Chapter 4

Coding Standards

4.1 Purpose

Consistent coding standards improve code readability, simplify debugging, reduce technical debt, and encourage maintainable software development.

4.2 General Principles

Development should prioritize:

- Readability
- Simplicity
- Maintainability
- Performance
- Reusability

4.3 Code Style

The following standards should be followed:

- Meaningful variable names
- Small focused functions
- Single responsibility principle
- Minimal global state
- Consistent indentation

4.4 Documentation

Every major class, function, and gameplay system should include technical documentation describing its purpose, inputs, outputs, and expected behavior.

4.5 Version Control

Git is used for version control.

Development should follow:

- Feature branches
- Frequent commits
- Clear commit messages
- Pull request reviews (for team development)

4.6 Summary

Following consistent coding standards improves software quality and enables sustainable long-term development.

Chapter 5

Core Gameplay Systems

5.1 Overview

The gameplay experience of *ECHO: The Living World* is built upon several interconnected systems. Each system operates independently while exchanging information through well-defined interfaces.

5.2 Primary Systems

The primary gameplay systems include:

- Player Controller
- Camera System
- Combat System
- Dialogue System
- Quest System
- Inventory System
- Companion System
- AI System
- Reputation System
- Save System

5.3 System Interaction

Gameplay systems exchange information using events, managers, and shared game state rather than direct dependencies wherever possible.

This architecture minimizes coupling while improving flexibility and maintainability.

5.4 Design Goals

Each gameplay system should satisfy the following objectives:

- Independent implementation
- High performance
- Easy debugging
- Expandability
- Reusability

5.5 Summary

The modular gameplay architecture enables rapid development while supporting future gameplay expansion and long-term maintainability.

Chapter 6

Player System

6.1 Overview

The Player System manages all functionality directly related to the player character. It acts as the central controller for movement, camera interaction, player state, attributes, equipment, and interactions with the game world.

The system is designed using a modular architecture, allowing each subsystem to operate independently while communicating through clearly defined interfaces.

6.2 Responsibilities

The Player System is responsible for:

- Character movement
- Camera control
- Player input processing
- Character attributes
- Equipment management
- Interaction with world objects
- Health and stamina management
- Player progression

6.3 Subsystems

The Player System consists of the following modules:

- Input Manager
- Movement Controller
- Camera Controller
- Attribute Component
- Equipment Component
- Interaction Component
- Animation Controller

6.4 Player States

The player may exist in several gameplay states:

- Idle
- Walking
- Running
- Sprinting
- Jumping
- Falling
- Climbing
- Swimming
- Dodging
- Attacking
- Blocking
- Interacting
- Dead

State transitions are managed through a finite state machine to ensure predictable behavior.

6.5 Extensibility

Future gameplay mechanics should be implemented as independent components without modifying the core Player Controller whenever possible.

6.6 Summary

The Player System serves as the foundation for player interaction, ensuring responsiveness, scalability, and maintainability throughout development.

Chapter 7

AI System

7.1 Overview

The AI System controls the behavior of all non-player characters (NPCs), enemies, wildlife, and companions. It provides decision-making, navigation, combat behavior, and environmental interactions.

7.2 Architecture

The AI framework consists of:

- AI Controller
- Behavior Tree
- Blackboard
- Perception System
- Navigation System
- Decision Layer

7.3 AI Categories

Different AI profiles are implemented for:

- Friendly NPCs
- Hostile Enemies
- Wildlife

- Bosses
- Companions
- Neutral Characters

7.4 Perception

AI entities perceive the environment using:

- Sight
- Hearing
- Damage Detection
- Environmental Triggers

7.5 Behavior

Typical AI behaviors include:

- Patrol
- Investigate
- Chase
- Attack
- Retreat
- Search
- Idle

7.6 Summary

The AI System is modular, allowing behaviors to be reused and extended across different character types.

Chapter 8

Combat System

8.1 Overview

The Combat System governs all offensive and defensive interactions between characters. It provides melee combat, ranged combat, damage calculation, hit detection, and defensive mechanics.

8.2 Components

Core combat components include:

- Weapon System
- Damage Calculator
- Hit Detection
- Animation System
- Targeting System
- Status Effect Manager

8.3 Combat Actions

Supported actions include:

- Light Attack
- Heavy Attack
- Charged Attack

- Block
- Dodge
- Counter
- Special Ability

8.4 Damage Types

Damage categories include:

- Physical
- Fire
- Ice
- Lightning
- Poison
- Echo Energy

8.5 Design Goals

The combat system should provide:

- Responsive controls
- Clear player feedback
- Balanced progression
- Extensible weapon support

8.6 Summary

Combat mechanics are implemented through modular components to simplify balancing and future expansion.

Chapter 9

Inventory System

9.1 Overview

The Inventory System manages all collectible items, equipment, crafting materials, consumables, and quest-related objects.

9.2 Inventory Categories

Items are organized into:

- Weapons
- Armor
- Consumables
- Crafting Materials
- Quest Items
- Key Items
- Currency

9.3 Operations

The system supports:

- Add Item
- Remove Item
- Equip Item

- Unequip Item
- Stack Items
- Sort Inventory
- Transfer Items

9.4 Data Storage

Each inventory item stores:

- Item ID
- Name
- Category
- Quantity
- Weight
- Rarity
- Description

9.5 Summary

The Inventory System is data-driven, enabling rapid content creation and efficient item management.

Chapter 10

Quest System

10.1 Overview

The Quest System manages all story missions, side quests, world events, and dynamic objectives throughout the game.

10.2 Quest Types

Supported quest categories include:

- Main Story
- Side Quest
- Companion Quest
- Exploration Quest
- Dynamic Event
- World Activity

10.3 Quest Structure

Each quest contains:

- Quest ID
- Title
- Description
- Objectives

- Rewards
- Prerequisites
- Completion Conditions

10.4 Quest States

A quest may exist in one of the following states:

- Locked
- Available
- Accepted
- In Progress
- Completed
- Failed

10.5 Design Goals

The Quest System should support branching objectives, dynamic world changes, and future content expansion while remaining data-driven.

10.6 Summary

The Quest System provides a flexible framework for narrative progression and player-driven exploration.

Chapter 11

Dialogue System

11.1 Overview

The Dialogue System manages all conversations between the player and non-player characters (NPCs). It supports story progression, side quests, branching conversations, reputation-based responses, and dynamic world interactions.

The system is designed to separate dialogue content from gameplay logic, enabling writers and designers to modify conversations without changing source code.

11.2 Core Components

The Dialogue System consists of the following components:

- Dialogue Manager
- Conversation Database
- Dialogue UI
- Choice Manager
- Condition Evaluator
- Event Trigger System

11.3 Dialogue Flow

A typical dialogue sequence follows these steps:

1. Player initiates interaction.
2. Dialogue Manager loads the conversation.

3. Conditions are evaluated.
4. Appropriate dialogue node is displayed.
5. Player selects a response.
6. Dialogue events are executed.
7. Conversation continues or ends.

11.4 Dialogue Features

The system supports:

- Branching conversations
- Multiple dialogue choices
- Reputation-based responses
- Quest integration
- NPC relationship tracking
- Cinematic dialogue sequences

11.5 Design Goals

The Dialogue System should provide:

- Data-driven conversations
- Easy localization
- Expandable dialogue trees
- Seamless gameplay integration

11.6 Summary

The Dialogue System enables immersive storytelling while remaining scalable, maintainable, and independent of gameplay implementation.

Chapter 12

Save and Persistence

12.1 Overview

The Save and Persistence System records and restores the player's progress throughout the game. It ensures continuity across gameplay sessions while maintaining data integrity and efficient storage.

12.2 Saved Data

The system stores:

- Player statistics
- Inventory
- Equipment
- Quest progress
- World state
- NPC relationships
- Reputation values
- Player location
- Game settings

12.3 Save Types

Supported save methods include:

- Manual Save
- Automatic Save
- Checkpoint Save

12.4 Persistence Strategy

Persistent world data includes:

- Opened containers
- Completed events
- Defeated enemies
- World choices
- Dynamic environment changes

12.5 Data Integrity

To prevent corruption, the system should:

- Validate save files
- Use version numbers
- Support backward compatibility
- Create backup saves

12.6 Summary

The Save and Persistence System provides reliable storage while supporting long-term progression and future game updates.

Chapter 13

Performance and Optimization

13.1 Overview

Performance optimization is essential to ensure stable gameplay, smooth frame rates, and efficient resource utilization across supported hardware.

Optimization should be considered throughout development rather than only during the final stages.

13.2 Performance Goals

Target objectives include:

- Stable frame rates
- Efficient memory usage
- Reduced loading times
- Low CPU overhead
- Efficient GPU utilization

13.3 Optimization Techniques

The project should utilize:

- Level Streaming
- Level of Detail (LOD)
- Occlusion Culling

- Asset Compression
- Object Pooling
- Asynchronous Asset Loading

13.4 Profiling

Performance should be monitored using:

- Unreal Insights
- Stat Commands
- Memory Profiler
- GPU Visualizer

13.5 Design Goals

Optimization decisions should prioritize maintaining gameplay quality while minimizing resource consumption.

13.6 Summary

Continuous profiling and optimization ensure that the game remains responsive and scalable throughout development.

Chapter 14

Future Expansion

14.1 Overview

The technical architecture is designed to accommodate future gameplay features, additional content, and long-term project growth without requiring major architectural changes.

14.2 Expansion Opportunities

Potential future additions include:

- New continents
- Additional factions
- Expanded questlines
- New combat abilities
- Additional companions
- Crafting system enhancements
- Dynamic weather improvements
- Online multiplayer support

14.3 Scalability Principles

The architecture follows these principles:

- Modular systems

- Data-driven configuration
- Loose coupling
- Reusable components
- Event-driven communication

14.4 Long-Term Vision

Future development should maintain compatibility with existing systems while allowing new features to integrate with minimal disruption.

14.5 Summary

Scalable engineering practices ensure that *ECHO: The Living World* can continue evolving throughout its development lifecycle.

Chapter 15

Appendix

15.1 Abbreviations

Term	Definition
AI	Artificial Intelligence
GDD	Game Design Document
TDD	Technical Design Document
NPC	Non-Player Character
UI	User Interface
HUD	Heads-Up Display
LOD	Level of Detail
FSM	Finite State Machine
API	Application Programming Interface
UML	Unified Modeling Language

15.2 Referenced Documents

- Game Design Document
- Narrative Bible
- Character Bible
- Art Bible
- Programming Guide

15.3 Glossary

Echo Energy The primary mystical force within the world of Aethera.

Game Manager The central controller responsible for coordinating high-level gameplay systems.

Behavior Tree A hierarchical AI decision-making structure used for NPC behavior.

Blackboard A shared data structure used by AI systems to store and access runtime information.

Data-Driven Design An approach where game content is defined through external data rather than hard-coded logic.

15.4 Document History

Version	Date	Description
1.0	July 21, 2026	Initial release of the Technical Design Document.